ROTA VITAL

GESTÃO DE HEMOCOMPONENTES

DESAFIO DO PI · INFRAESTRUTURA DE SOFTWARE · 2026.2

Container, TDD e Deploy automatizados

CESAR School · Prof. Raoni Monteiro de Oliveira

INTEGRANTES

Victor José Paes e Silva (líder)
Eduardo de Souza Cavalcanti Junior
Felipe Franca Alves de Lima
Helamã Leone de Lima Procídio
João Pedro Arruda Guimarães
Júlia Oliveira Veríssimo
Lucas Paguetti Pereira
Tiago Luiz Moreira de Vasconcelos

O DESAFIO

Uma pergunta, três entregas

> “Se um colega clonar o repositório amanhã, ele consegue rodar, testar e publicar o sistema sem pedir ajuda a ninguém?”

E1 · 3,0 pts

Container

Sobe do zero com um comando

E2 · 3,0 pts

TDD

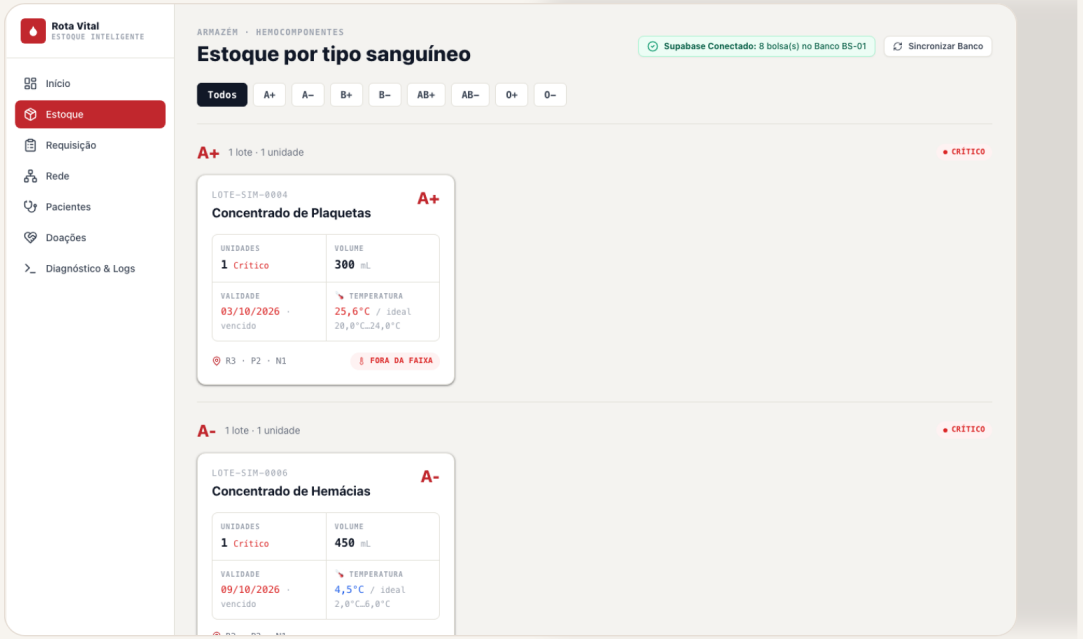
A regra nasce do teste

E3 · 3,0 pts

Deploy

Merge na main vai sozinho ao ar

Uma apoia a outra: a esteira do E3 usa o Dockerfile do E1 e só publica se os testes do E2 passarem.



O Rota Vital: estoque de sangue entre bancos e hospitais, onde um erro de regra chega ao paciente.

E1 · CONTAINER AUTOMATIZADO

O sistema sobe com um comando

```
# na raiz do repositório
$ git clone …/viictorpaes/Rotavital
$ cp .env.example .env
$ docker compose up -d --build
$ docker compose ps
db, backend e frontend: (healthy)
```

`localhost → site` `localhost:8080 → API`

Só Git e Docker

Java, Maven, Node e Postgres ficam dentro dos containers.

O build roda os testes

O 1º estágio do Dockerfile roda os 208 testes unitários: se um falha, não sai imagem.

Nenhum segredo no Git

A senha fica no .env, fora do Git. Sem DB_PASSWORD, o Compose nem sobe.

E1 · COMPOSE.YAML

Três containers que se acham pelo nome

Navegador

Abre localhost, na porta 80 da máquina.

frontend

Nginx sem root com o build do React; repassa /api/ ao backend.

`GET /`

backend

API Spring Boot, imagem de 2 estágios; conecta em db:5432.

`health = UP`

db

Postgres 17 criado com as migrations do Supabase e o seed.

`pg_isready`

Ordem de subida

db saudável → backend saudável → frontend (healthcheck + depends_on).

Rede do Compose

Os serviços se acham pelo nome; o banco não publica porta para fora.

Volume nomeado

dados-postgres: os dados sobrevivem ao docker compose down.

E1 · EVIDÊNCIAS

Subiu do zero, testado e saudável

```
$ docker compose up -d --build
[INFO] Tests run: 208, Failures: 0, Errors: 0
[INFO] BUILD SUCCESS
$ docker compose ps
backend: Up 27 seconds (healthy)
db: Up 32 seconds (healthy)
frontend: Up 16 seconds (healthy)
$ curl localhost:8080/actuator/health
{"status":"UP"}
```

Saídas reais do terminal (14 e 18/10/2026, macOS + Docker Desktop), resumidas.

378 MB

imagem da API: só o JRE, usuário sem root

91 MB

imagem do site: Nginx sem root

Volume testado

Reservamos uma bolsa, derrubamos com down e subimos de novo: ela continuou RESERVADA.

E2 · TDD

Story map: onde a regra machuca mais

	Entrar	Painel	Estoque	Requisitar	Transportar	Atender	Doar
Backbone	entrar com o perfil	ver alertas	ver lotes	pedir bolsa	planejar a rota	concluir o atendimento	agendar doação
Já existia	login (HU-01)	totais por tipo	consulta (HU-03)	—	rota (HU-07)	—	telas (HU-09/10)
Release 1 (TDD)	—	HU-02 alertas	HU-03 cenários	HU-04 sugestão de lote	—	HU-08 baixa da bolsa	—
Release 2	bloquear por perfil	notificar alerta	recebimento	gravar alocação	rastreio ao vivo	baixa ligada ao paciente	doador apto

3 histórias da Release 1 → 13 tarefas. Cada tarefa de regra tem teste unitário; cada história, teste de integração e de aceitação.

E2 · POR QUE ESSAS PRIMEIRO

ABO/Rh primeiro: um erro ali põe um paciente em risco.

1 **Compatibilidade ABO/Rh**
Reação hemolítica. E o código só aceitava o mesmo tipo.

2 **Bolsa apta; vencida só se descarta**
Bolsa vencida ou fora da temperatura chega ao paciente.

3 **Baixa e transições de status**
O estoque mente: bolsa usada duas vezes.

4 **FEFO + mesmo tipo primeiro**
Desperdício: bolsa vence na prateleira, O− gasto à toa.

5 **Alertas do painel**
É aviso, não bloqueio: as regras acima já impedem o uso.

E2 · HISTÓRICO RED → GREEN

Teste vermelho antes, código depois

Tarefa	test: o teste falha	feat: o código passa
T4.1 · ABO/Rh das hemácias	67d4c55 · 9 de 10 falham	1b9cd69 · 69/69
T4.2 · plasma e crio	9baea07 · 8 falham	cbbd912 · 93/93
T4.3 · bolsa apta	e901034 · 3 de 9 falham	21c2312 · 98/98
T4.4 · ordem FEFO	8c9fb6b · 7 de 8 falham	e4d884b · 106/106
T4.5 · API da sugestão	2dcd504 · 404 nos 6 cenários	547a00f · cenários verdes
T2.1–T2.4 · alertas	93980fa · 9 de 11 falham	5baca27 · 117/117
T2.5 · API dos alertas	b82afdb · 404 nos 2 cenários	591006a · cenários verdes
T8.1–T8.2 · transições	60207da · 29 de 40 falham	1dd60ba · 157/157
T8.3 · baixa no Postgres	ec1c83d · 405 em 7 testes	f2d1e0f · 157 + 20 verdes

9 pares de commits. O PR vai com “Create a merge commit”: o squash apagaria esse histórico.

E2 · TESTES POR NÍVEL

Três níveis de teste, todos na esteira

208

unitários · mvn test

Cada regra, sem banco: JUnit 5, repositório falso e MockMvc. Rodam também no Dockerfile.

7

integração · mvn verify

API + banco num Postgres 17 de verdade (Testcontainers), com as migrations do Supabase.

13

aceitação · mvn verify

Cenários Gherkin em português, chamando a API por HTTP: o médico consegue ler.

```
Cenário: Nunca sugere hemácias Rh+ para paciente Rh-
  Quando peço 3 unidades de hemácias para um paciente "O-" no banco "BS-01"
  Então o sistema sugere os lotes "CH-1042, CH-1043"
  E a sugestão avisa que faltam 1 unidade
```

E3 · PIPELINE NO GITHUB ACTIONS

A esteira só publica com tudo verde

push ou PR
Em qualquer branch.

→

testes + build
mvn verify e npm run build.

→

imagens Docker
Os Dockerfiles do E1; na main, a API vai ao GHCR.

→

deploy + smoke
Só na main: API no Render, site na Vercel.

Esteira CI/CD #2 · sucesso — f2d1e0f · 14/10/2026

✓ Backend · testes	55 s
✓ Frontend · build	15 s
✓ Imagens Docker	1 min 18 s
○ Deploy · Render	pulado: não é a main
○ Rollback · Render	pulado: só manual

Execução real na branch feature/container-tdd-deploy, com os dados da API do GitHub.

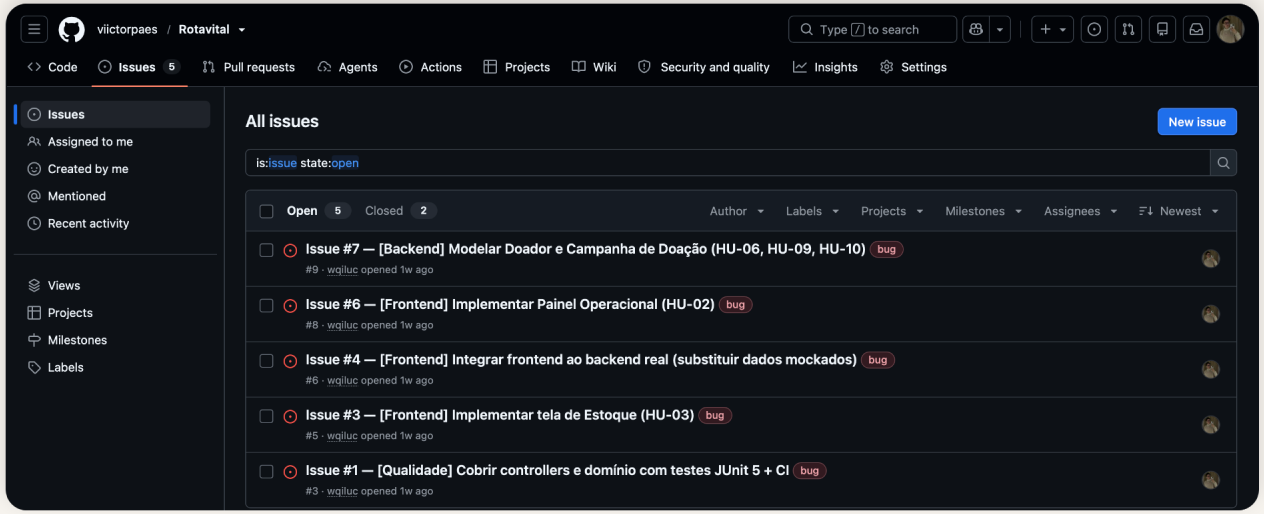
Os gates

- Teste vermelho para tudo antes da imagem.
- Deploy só em push na main, ou seja, no merge de um PR.
- O smoke test espera o sha novo no ar e o health UP.

E3 · FLUXO, SEGREDOS E ROLLBACK

GitHub Flow com a main protegida

issue → branch → pull request → esteira verde → merge na main → deploy



As issues do repositório viram branches e PRs (print do GitHub).

Por que GitHub Flow

Grupo pequeno, uma versão no ar e deploy a cada merge. Os PRs #17 a #28 já seguiam isso.

Segredos no cofre

Senhas no .env local, no Render e no environment producao do GitHub. Nada no repositório.

Rollback em minutos

Run workflow com o sha anterior: volta a imagem do GHCR e o site daquele sha.

A TABELA DE DECISÕES DO README

Decisões, uma linha cada

Decisão	Escolha	Por quê
Imagem base	Temurin 21 JRE Alpine · Nginx sem root	Menor, com menos vulnerabilidades e sem root
Multi-stage	1º estágio compila e testa; 2º leva o .jar	Produção não carrega Maven nem JDK
Dados	Volume nomeado dados-postgres	Sobrevivem ao docker compose down
Rede	Rede do Compose, pelo nome do serviço	Sem IP fixo; o banco não publica porta
Testes	Testcontainers + Cucumber	Postgres de verdade; cenários que o médico lê
CI/CD	GitHub Actions + GHCR, tag = sha	O código já está no GitHub; rollback exato
Fluxo	GitHub Flow	Grupo pequeno, deploy a cada merge
Deploy	Render (API) + Vercel (site) + Supabase	Grátis; a imagem no Render, o site na CDN
Segredos	.env local · Render · GitHub Environments	Nada no repositório

JORNADA TDD E USO DE IA

O que o TDD e a IA ensinaram

1 O story map mostrou onde testar

Em cada atividade, a regra que mais machuca se der errado.

2 O teste expôs um gap

O README prometia ABO/Rh, mas o código só aceitava o mesmo tipo.

3 Ver o teste falhar

Nos 9 pares, o teste falhou pelo motivo certo antes de passar.

4 Banco de verdade nos testes

O Postgres das migrations pegou constraints que um mock não pega.

Uso de IA, declarado

O Claude Code rascunhou Dockerfiles, esteira, testes e README. O grupo rodou, conferiu e assina.

Erros que só apareceram rodando: script num bind mount do macOS, ${var,,} no bash, sha curto no rollback.

Os prompts principais estão no README.

STATUS DA ENTREGA

Onde estamos

Pronto

- E1: sobe do zero com um comando, 3 serviços saudáveis
- E2: 9 pares red → green; 208 + 7 + 13 testes na esteira
- E3: esteira verde na branch; deploy e rollback no workflow
- README com as decisões e o uso de IA

Falta (contas do grupo)

- Proteger a main: PR e checks obrigatórios
- Ligar Render, Vercel e cadastrar os segredos
- Prints da execução vermelha, da verde e do rollback
- URL pública no ar

Obrigado! `github.com/viictorpaes/Rotavital`